

# Studying the Footprints of AI Coding Agents in Blockchain Repositories

Munim Iftikhar  
Lahore University of Management  
Sciences, Lahore  
Lahore, Pakistan  
24030026@lums.edu.pk

Maaz Shahid  
Lahore University of Management  
Sciences, Lahore  
Lahore, Pakistan  
27100445@lums.edu.pk

Shahreyar Ashraf  
Lahore University of Management  
Sciences, Lahore  
Lahore, Pakistan  
26100342@lums.edu.pk

Muhammad Saqib Ilyas  
Lahore University of Management  
Sciences, Lahore  
Lahore, Pakistan  
saqibm@lums.edu.pk

Abdul Ali Bangash  
Lahore University of Management  
Sciences, Lahore  
Lahore, Pakistan  
abdulali@lums.edu.pk

## Abstract

AI coding agents are rapidly transforming software development, yet researchers do not fully understand their effectiveness in specialized domains. Previous studies examined how these agents contribute code via pull requests (PRs) across repositories from various domains, making it difficult to isolate domain-specific patterns. We analyze 497 agent-contributed PRs from 162 blockchain-focused open-source GitHub repositories in the AIDev dataset. We categorize agents into high-contribution agents and low-contribution agents groups based on their activity levels in blockchain repositories. Our analysis reveals that both groups have nearly identical PR acceptance rates (57.8% vs. 56.2%), but high-contribution agents get their PRs merged 15 hours faster. For PR size (we divided PRs to four quartiles based on total number of codes lines changed), we find no difference between the PR acceptance rate based on their code churn rate, however, we observe an increase in PR resolution time by their size, with new features (136 increase), bug fixes (132 increase), and documentation (40 hours increase) showing the strongest sensitivity to PR size. We make our replication package publicly available [4].

## CCS Concepts

• **Software engineering** → **Agentic software engineering**.

## Keywords

AI coding agents, blockchain, pull request analysis, mining software repositories

### ACM Reference Format:

Munim Iftikhar, Maaz Shahid, Shahreyar Ashraf, Muhammad Saqib Ilyas, and Abdul Ali Bangash. 2026. Studying the Footprints of AI Coding Agents in Blockchain Repositories. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793598>



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2474-9/26/04  
<https://doi.org/10.1145/3793302.3793598>

## 1 Introduction

AI coding agents, such as GitHub Copilot, Devin, Cursor, and Codex, have fundamentally changed how developers write and review code. As organizations integrate these tools into development workflows, researchers must understand their effectiveness in specialized domains. Blockchain development presents a compelling case study because it combines security-critical code where vulnerabilities cost millions [15], domain-specific languages underrepresented in training data, and complex cryptographic implementations requiring deep expertise.

Prior work examines either general-agent performance across mixed repositories [12, 17] or blockchain development, without considering AI contributions [11]. This gap should be examined because blockchain's unique characteristics may amplify or mitigate patterns observed in general repositories. A replication package containing all data and analysis scripts is available at [20]. We address this through analysis of agent-contributed PRs specifically in blockchain contexts. Specifically, we focus on the following research questions:

**RQ1: How do PR acceptance rates and PR resolution times differ between high-contribution and low-contribution agents?**

We categorized agents into two groups based on their contribution volume and compared their PR outcomes.

**RQ2: Does the PR acceptance rate and PR resolution time differ by PR size?** We categorized PRs into four PR size quartiles based on lines of code changed and analyzed their outcomes.

## 2 Related Work

Das et al. [11] analyzed 3,664 blockchain repositories using multi-source keyword extraction from CoinMarketCap, permissioned blockchain platforms, and blockchain-specific terms. They found organizations dominate blockchain repositories (58.6%) and tool repositories show higher activity than applications. Trockman et al. [18] conducted an econometric study of 241 cryptocurrencies on GitHub over one year, finding that repository popularity (stars) correlates with higher market capitalization, while development activity metrics show limited predictive power. Vaccargiu et al. [19]

examined 10 Ethereum repositories (129, 884 commits, 40, 550 issues) over 10 years, finding technical events prompt increased pre-empt activity while market events lead to reactive development. Unlike these studies examining general development patterns or market correlations, we analyze AI agent contributions across the blockchain ecosystem, extending Das et al.'s [11] keyword approach to identify agent-contributed PRs.

Grewal et al. [12] found 54% of ChatGPT-generated code is incorporated unchanged, with modifications typically occurring within a day. Silva et al. [17] examined 155 ChatGPT-assisted PRs, identifying four request categories where code generation required significantly more prompts than other tasks. These studies examine ChatGPT usage across general repositories. We extend this work by analyzing multiple AI agents specifically in blockchain repositories, introducing contribution-based agent categorization, and analyzing complete PR lifecycles.

### 3 Methodology

Figure 1 illustrates our methodology diagram, which consists of three phases: data collection and blockchain repository identification, RQ1 analysis comparing contribution groups, and RQ2 analysis examining PR size differences.

#### 3.1 Data Collection

We use the AIDev dataset (version July 2025) [13] containing 33, 596 agent-contributed PRs across 116, 211 repositories from five agents: Codex, Copilot, Devin, Cursor, and Claude Code.

**Multi-Source Keyword Collection.** We collected blockchain-related keywords from three sources: (1) API extraction using CoinMarketCap, CoinGecko, and Coinbase APIs; (2) manual extraction from blockchain infrastructure documentation websites, including Alchemy, QuickNode, and ChainLink [1–3, 5–10, 16]; and (3) curated blockchain keywords generated via Claude and verified by one researcher. This yielded 1, 719 initial keywords. Two of the researchers further manually refined the initial keywords by removing generic and merging the ones that were relevant, finalizing a list of 1, 845 final keywords.

**Repository Extraction and Filtering.** Running 1, 845 final keywords against the AIDev dataset repositories with  $\geq 3$  stars yield 162 blockchain repositories containing 502 PRs.

#### 3.2 Agent Categorization by Contribution Level

A key methodological decision in our study is how we group agents for comparison. Rather than comparing each agent individually (which would give us 5 groups with very unequal sample sizes), we categorize agents into two groups based on how many PRs they contributed to blockchain repositories.

The number of PRs each agent contributed is 230 (Codex), 137 (Devin), 109 (Copilot), 17 (Cursor), and 4 (Claude Code). We calculated the average number of contributions across agents, excluding Claude Code as an outlier (only 4 PRs). The average threshold was 123 PRs. Agents contributing more than this mean were classified as "high-contribution" agents; those contributing fewer were classified as "low-contribution" agents.

This grouping approach has two advantages: (1) it creates more balanced sample sizes for statistical comparison, and (2) it allows

**Table 1: Distribution of PRs by category**

| Category                                            | Number of PRs |
|-----------------------------------------------------|---------------|
| new feature (new feature or functionality addition) | 144           |
| bug fixes                                           | 123           |
| infrastructure                                      | 113           |
| code refactor (code restructuring)                  | 56            |
| documentation                                       | 24            |
| test (Adding or modifying tests)                    | 20            |
| frontend (UI/UX, whitespace, or style changes)      | 17            |
| others                                              | 5             |

us to ask whether agents that are heavily used in blockchain development perform differently from those that are rarely used.

#### 3.3 PR Categorization

**Topic Categories.** We adopt the methodology of Zhao et al. [20] to classify PRs using large-language models. We used Claude 3.5 Sonnet to classify each PR based on its title and body into one of the eight categories out of ten categories which are defined for the AIDev dataset PRs by Li et al. [14]. Due to limited data in some categories, we merged CI/CD, build system changes, dependency updates, maintenance tasks, and performance optimizations to the "infrastructure" PR topics category. For blockchain repositories, we defined "smart contract features" to "new feature", "contract bugs" to "bug fixes", "gas optimization" to "infrastructure", and "smart contract tests" to "test", which was not catered by Li et al. [14]. Final categories are given in 1.

We excluded PRs in the "other" category (5 PRs) from our blockchain PRs dataset due to insufficient sample sizes, resulting in a total of 497 PRs across five agents for analysis.

**PR Size Categories.** We classify PRs based on their lines of code (LOC) modified using data from AIDev's `pr_commit_details` table. We divided PRs into four quartiles based on the distribution of lines changed in our dataset: **Q1:**  $\leq 34$  lines changed (bottom 25%); **Q2:** 34–159 lines changed (25th–50th percentile); **Q3:** 159–776 lines changed (50th–75th percentile); **Q4:**  $> 776$  lines changed (top 25%).

### 4 Results

#### 4.1 RQ1: Differences by agents contribution

**Motivation:** We analyze whether agents that frequently contribute to blockchain repositories perform differently from those that rarely do. Our insights can help teams select appropriate agents and understand what factors influence PR outcomes.

**Approach:** We calculated the PR acceptance rate as the percentage of PRs merged by maintainers. We also measured PR resolution time as the hours between PR creation and merge (for merged PRs only). We compared these metrics between high-contribution agents (Codex, Devin) and low-contribution agents (Copilot, Cursor, Claude Code) using Chi-Square tests for PR acceptance rates and Mann-Whitney U tests for PR resolution times. For PR category-wise comparison, we apply the Kruskal-Wallis test for PR resolution times.

**Findings: Both groups have nearly identical PR acceptance rates.** High-contribution agents had 212 of 367 PRs accepted (57.8%), while low-contribution agents had 73 of 130 PRs accepted (56.2%)—a difference of only 1.6 percentage points. Across both groups, 285

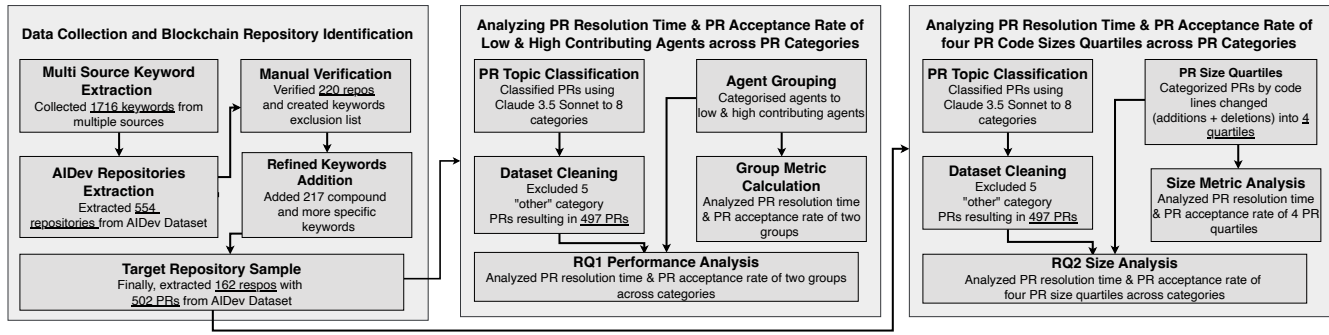


Figure 1: Methodology pipeline showing our three-phase approach.

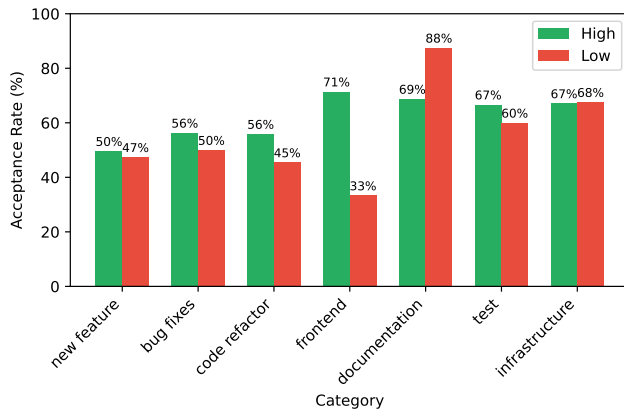


Figure 2: A boxplot-based PR acceptance rates comparison of agents' contribution groups (low/high) across PR topic categories.

of 497 total PRs were accepted (57.3%). This difference is not statistically significant ( $\chi^2 = 0.026$ ,  $p > 0.05$ ).

**High-contribution agents get PRs merged 15 hours faster.** Among merged PRs, high-contribution agents ( $n = 212$ ) have a median PR resolution time of 3.2 hours with a mean of 52.7 hours and an IQR of 0.3–37.4 hours. Low-contribution agents ( $n = 73$ ) have a median PR resolution time of 18.5 hours with a mean of 57.7 hours and an IQR of 3.6–68.7 hours. This makes high-contribution agents an absolute 15 hours faster (3.2 – 18.5) than low-contribution agents. This difference is statistically significant ( $p < 0.05$ ).

**These patterns do not vary by PR topics category.** We examined whether differences vary by PR type (bug fix, new feature, etc.). Figure 2 shows PR acceptance rates by category for each contribution group. Neither group shows statistically significant variation across categories (high:  $\chi^2 = 8.61$ ,  $p > 0.05$ ; low:  $\chi^2 = 7.99$ ,  $p > 0.05$ ).

Similarly, PR resolution times do not significantly differ across categories within either group. Figure 3 shows median PR resolution times by category and contribution group. High-contribution agents consistently achieve faster resolution across most categories, with frontend PRs being the only exception (59.2h vs. 0.9h for low-contribution agents).

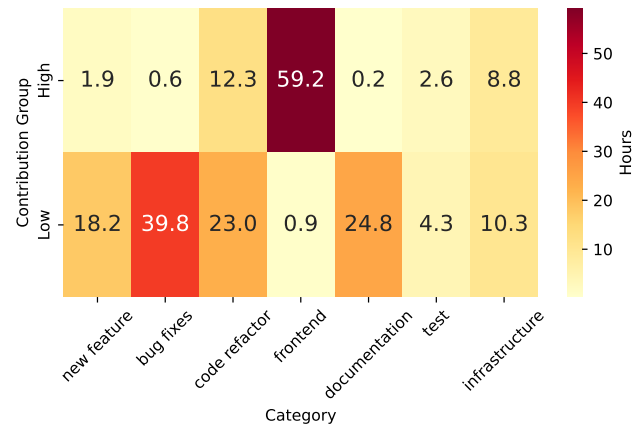


Figure 3: A heatmap-based median PR resolution time (hours) comparison of agents' contribution groups (low/high) across PR topic categories.

**RQ1 Summary:** Both high-contribution and low-contribution agents achieve similar acceptance rates (57%). However, high-contribution agents get PRs merged 15 hours faster. These patterns remain consistent across all PR topic categories.

## 4.2 RQ2: Differences by PR size

**Motivation:** By investigating how outcomes differ by PR size, we seek to understand whether maintainers treat small and large contributions differently, and provide guidance on optimal PR sizing.

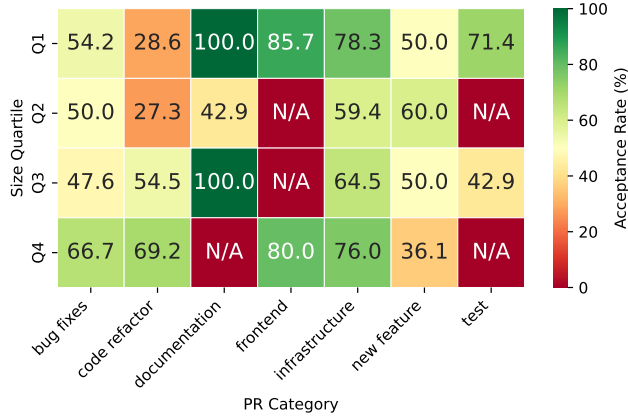
**Approach:** We categorized PRs into four PR size quartiles based on lines of code changed: Q1 ( $\leq 34$  LOC), Q2 (34–159 LOC), Q3 (159–776 LOC), and Q4 ( $> 776$  LOC). We compared PR acceptance rates using Chi-Square tests and PR resolution times using Kruskal-Wallis tests across quartiles. We also applied bonferroni correction to the p-values.

**Findings: PR acceptance rate does not differ by PR size.** Table 2 shows PR acceptance rates range from 54.0% to 62.6% across quartiles, but these differences are not statistically significant ( $\chi^2 = 2.58$ ,  $p > 0.05$ ). Figure 4 shows the breakdown by PR size quartile across PR categories, confirming no consistent pattern.

**PR resolution time differs significantly by PR size.** Small PRs (Q1) have a median PR resolution time of 0.5 hours (about 30 minutes), while Very Large PRs (Q4) take 39.4 hours—an 82×

**Table 2: PR Acceptance rates and PR resolution times by PR size quartile.**

| Size             | # PRs | Merge Rate | Med. Time | Mean Time |
|------------------|-------|------------|-----------|-----------|
| Q1 ( $\leq 34$ ) | 123   | 62.6%      | 0.5h      | 12.2h     |
| Q2 (34–159)      | 124   | 54.0%      | 3.8h      | 36.7h     |
| Q3 (159–776)     | 122   | 54.9%      | 20.2h     | 52.1h     |
| Q4 ( $> 776$ )   | 123   | 60.2%      | 39.4h     | 114.7h    |

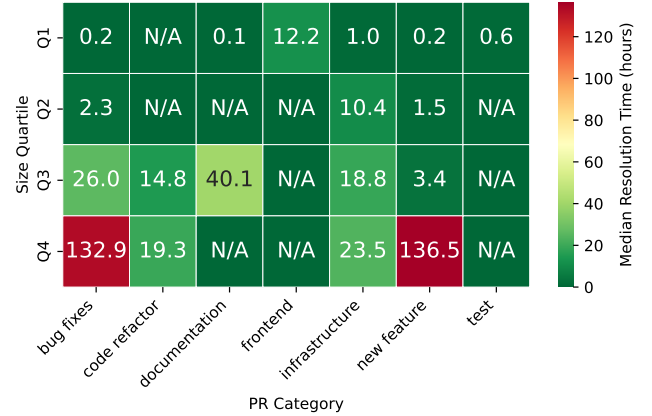
**Figure 4: A heatmap-based PR acceptance rate comparison of PR Quartiles (by PR size) across PR categories. N/A indicates insufficient data. Numbers in cells represent sample sizes.**

increase. This difference is highly significant ( $p < 0.05$ ). Pairwise comparisons show that small PRs resolve significantly faster than all other sizes.

**Category-wise, resolution time varies significantly within each PR size quartile.** Figure 5 shows median PR resolution times by PR size quartile across PR topics category (Q1:  $p < 0.05$ , Q2:  $p > 0.05$ , Q3:  $p > 0.05$ , Q4:  $p < 0.05$ ). Q3 shows no significant difference across categories, but Q1, Q2, and Q4 shows significant differences across categories. The most striking finding is category-dependent scaling in PR resolution time: **documentation are most sensitive** with 0.1h (Q1) to 40.1h (Q3)—an absolute increase of 40 hours, **new feature shows second most sensitivity** with 0.2h (Q1) to 136.5h (Q4)—an absolute increase of 136 hours, **bug fixes hews third most sensitivity** with 0.2h (Q1) to 132.9h (Q4)—an absolute increase of 132 hours, **infrastructure hews second least sensitivity** with 1.0h (Q1) to 23.5h (Q4)—an absolute increase of 22 hours, and **refactoring is least sensitive** with 14.8h to 19.3h—only 3 hours increase.

Within Very Large PRs (Q4), new feature takes 113 hours longer PR resolution time than infrastructure PRs ( $p < 0.05$ ) and 117 hours longer PR resolution time than code refactor ( $p < 0.05$ ). Large new feature and bug fixes PRs require substantially more review effort.

**RQ2 Summary:** Maintainers accept small PRs at the same rate as large PRs. However, a 30-line PR gets merged in about 30 minutes while an 800+ line PR takes nearly two days. New feature and bug fixes resolution time changes significantly by size, while refactoring resolution time changes least in PR sizes.

**Figure 5: A heatmap based median PR resolution time (hours) comparison of PR Quartiles (by PR size) across PR topic categories. N/A indicates insufficient data.**

## 5 Discussion

**For Practitioners:** The choice between high and low-contribution agents does not differ in PR acceptance rates (both 57%). However, high-contribution agents achieve 15 hours faster merge times. Teams should keep PRs small—while large PRs have similar PR acceptance rates, they take dramatically longer to merge. Bug fixes and new feature are especially sensitive to size (132 – 136 hours increase), so consider breaking large bug fixes and features into smaller incremental PRs.

**For Researchers:** Our contribution looks into the characteristics of AI-based PRs in the blockchain domain. Future work could explore whether these patterns hold across other specialized domains and investigate the mechanisms behind faster PR resolution times for high-contribution agents.

### 5.1 Threats to Validity

**Internal:** LLM-based PR topics categorization may introduce classification errors. Small sample sizes in some category-size combinations limit statistical power. Agent grouping uses a single median threshold; different thresholds may yield different results. Results are specific to blockchain repositories and may not generalize to other domains. The AIDev dataset captures specific agents during a limited time period. PR acceptance rate treats all merges equally, though some require extensive revision. PR resolution time includes maintainer idle periods, not just active review time.

## 6 Ethical Considerations

This study did not require ethical approval since it did not involve any human subjects. We used the publicly available AIDev dataset [13] and properly cited its authors throughout the paper.

## 7 Conclusion

We present the first analysis of AI coding agent performance in blockchain repositories, examining 497 PRs across 162 repositories. Our contribution-based grouping approach reveals that while high-contribution agents (Codex, Devin) and low-contribution agents (Copilot, Cursor, Claude Code) have similar PR acceptance rates

(57%), high-contribution agents achieve 15 hours faster PR resolution times. PR acceptance rate is not significantly different in PR size but significantly differs by PR resolution time. This difference varies by category, with the new feature showing the strongest size sensitivity and refactoring the weakest. Our findings suggest that for blockchain development, teams should prioritize small PRs for faster integration and be especially careful with large bug fix PRs. The consistent PR acceptance rates across agent types indicate that maintainers evaluate contributions on their merits rather than their source.

## References

- [1] Alchemy. 2024. API Overview. <https://www.alchemy.com/docs/reference/api-overview> Accessed: December 2024.
- [2] Alchemy. 2024. List of dApps. <https://www.alchemy.com/dapps/list-of/> Accessed: December 2024.
- [3] Alchemy. 2024. Node Supported Chains. <https://www.alchemy.com/docs/reference/node-supported-chains> Accessed: December 2024.
- [4] Anonymous. 2025. Replication Package: Studying the Footprints of AI Coding Agents in Blockchain Repositories. <https://zenodo.org/records/18040009> Accessed: 2025-12-09.
- [5] Chainlink. 2024. CCIP Directory: Mainnet. <https://docs.chain.link/ccip/directory/mainnet> Accessed: December 2024.
- [6] Chainlink. 2024. CCIP Directory: Testnet. <https://docs.chain.link/ccip/directory/testnet> Accessed: December 2024.
- [7] Chainlink. 2024. Chainlink Documentation. <https://docs.chain.link/chainlink> Accessed: December 2024.
- [8] Chainlink. 2024. Chainlink VRF (Verifiable Random Function). <https://docs.chain.link/vrf> Accessed: December 2024.
- [9] Chainlink. 2024. Cross-Chain Interoperability Protocol Documentation. <https://docs.chain.link/ccre> Accessed: December 2024.
- [10] Chainlink. 2024. Faucets. <https://chain.link/faucets> Accessed: December 2024.
- [11] Ajoy Das, Gias Uddin, and Guenther Ruhe. 2022. An empirical study of blockchain repositories in github. In *Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering*. 211–220.
- [12] Balreet Grewal, Wentao Lu, Sarah Nadi, and Cor-Paul Bezemer. 2024. Analyzing developer use of chatgpt generated code in open source github projects. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 157–161.
- [13] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. AIDev: A Dataset of AI Teammates in Software Engineering. [https://github.com/SAILResearch/AI\\_Teammates\\_in\\_SE3](https://github.com/SAILResearch/AI_Teammates_in_SE3) Available at [https://github.com/SAILResearch/AI\\_Teammates\\_in\\_SE3](https://github.com/SAILResearch/AI_Teammates_in_SE3).
- [14] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003* (2025).
- [15] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 254–269.
- [16] QuickNode. 2024. Supported Chains. <https://www.quicknode.com/chains> Accessed: December 2024.
- [17] Julyanara R Silva, Carlos Eduardo C Dantas, and Marcelo A Maia. 2025. What developers ask to chatgpt in github pull requests? an exploratory study. *arXiv preprint arXiv:2508.17161* (2025).
- [18] Asher Trockman, Rijnard Van Tonder, and Bogdan Vasilescu. 2019. Striking gold in software repositories? An econometric study of cryptocurrencies on GitHub. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 181–185.
- [19] M. Vaccargiu, S. Aufiero, C. Ba, S. Bartolucci, R. Clegg, D. Graziotin, R. Neykova, R. Tonelli, and G. Destefanis. 2025. Mining a Decade of Event Impacts on Contributor Dynamics in Ethereum: A Longitudinal Study. In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR '25)*. ACM, New York, NY, USA. [https://figshare.com/articles/journal\\_contribution/Mining\\_a\\_Decade\\_of\\_Event\\_Impacts\\_on\\_Contributor\\_Dynamics\\_in\\_Ethereum\\_A\\_Longitudinal\\_Study\\_-\\_MSR\\_2025/27629559](https://figshare.com/articles/journal_contribution/Mining_a_Decade_of_Event_Impacts_on_Contributor_Dynamics_in_Ethereum_A_Longitudinal_Study_-_MSR_2025/27629559)
- [20] Zhimin Zhao, Yihao Chen, Abdul Ali Bangash, Bram Adams, and Ahmed E Hassan. 2024. An empirical study of challenges in machine learning asset management. *Empirical Software Engineering* 29, 4 (2024), 98.